K-means Paralelo con OpenMP

COMPUTO PARALELO Y EN LA NUBE - Primavera 2026

Rafael Harry Gomar Dawson

Adrián Janus González

Fecha: 24 de marzo de 2026

1. Introducción

Este documento presenta el desarrollo, la implementación y la evaluación experimental de una versión serial y una versión paralela del algoritmo K-means en lenguaje C. La versión paralela fue implementada con OpenMP para analizar si la paralelización permite obtener una mejora de rendimiento medible y reproducible frente a la versión serial.

K-means es un algoritmo de agrupamiento no supervisado ampliamente utilizado en análisis de datos y aprendizaje automático. Su objetivo es particionar un conjunto de puntos en k clústeres, asignando cada observación al centroide más cercano. Aunque su idea es simple, su costo computacional aumenta con el número de puntos, la dimensionalidad y las iteraciones requeridas para converger.

Desde la perspectiva del cómputo paralelo, K-means es relevante porque la fase de asignación de puntos puede distribuirse entre varios hilos. Sin embargo, la actualización de sumas y conteos por clúster introduce problemas de sincronización que deben resolverse cuidadosamente. Las pruebas se ejecutaron en Windows 11 Home sobre un Intel Core i7-1195G7 con 8 procesadores lógicos. Se generaron 700 registros experimentales validados. El mejor speedup global fue de 3.82 en 3D con N=1000000 y 16 hilos, mientras que en 2D el mejor resultado fue de 3.38. En general, los resultados muestran que la estrategia de acumuladores privados por hilo mejora el rendimiento de forma consistente en entradas medianas y grandes.

2. Objetivo general

El objetivo general del proyecto fue implementar el algoritmo K-means en lenguaje C en una versión serial y en una versión paralela con OpenMP, y comparar experimentalmente ambas soluciones para determinar el impacto de la paralelización sobre el rendimiento.

El problema de agrupamiento que aborda K-means consiste en encontrar una partición de los datos que minimice la distancia entre los puntos y el centroide de su grupo. En cada iteración, el algoritmo alterna dos operaciones: asigna cada punto al centroide más cercano y recalcula cada centroide como el promedio de los puntos asignados. Este ciclo se repite hasta que no haya cambios en las asignaciones, hasta que el desplazamiento de los centroides sea menor que una tolerancia predefinida o hasta alcanzar un máximo de iteraciones.

Desde el punto de vista del rendimiento, la fase crítica es la asignación. Para cada punto de entrada es necesario comparar su distancia respecto de todos los centroides, lo que convierte al algoritmo en un candidato natural para paralelización cuando el número de puntos es grande. No obstante, la ventaja real de usar varios hilos depende de si el trabajo útil supera el costo adicional de administrar el paralelismo.

3. Solución implementada

La solución fue desarrollada en C con una separación explícita entre la interfaz de línea de comandos, la lectura y escritura de archivos CSV, el núcleo compartido del algoritmo y los backends de ejecución serial y paralela. Esta organización permite conservar una única semántica de convergencia y aislar las diferencias de implementación al punto exacto donde realmente cambian: la asignación y acumulación por punto.

La arquitectura del proyecto responde a un criterio de claridad y mantenibilidad. El archivo principal orquesta la ejecución, procesa argumentos, carga datos y dispara el backend correspondiente. El módulo compartido del algoritmo implementa la validación del problema, la inicialización de centroides, la actualización de estos y el ciclo general de K-means. Por su parte, los módulos serial y OpenMP encapsulan únicamente la lógica de asignación y acumulación. Finalmente, la capa de CSV abstrae la entrada y la salida de datos, mientras que los scripts automatizan compilación, experimentación y síntesis de resultados.

3.1. Versión serial

La versión serial recorre secuencialmente todos los puntos, calcula la distancia al centroide más cercano y actualiza estructuras globales de sumas y conteos por clúster. Esta implementación funciona como baseline experimental, ya que representa el costo del algoritmo sin los gastos propios del paralelismo. Su principal utilidad dentro del proyecto es servir como punto de comparación justo para el cálculo de speedup.

Debido a que la versión serial comparte el mismo núcleo de convergencia con la versión paralela, cualquier diferencia de tiempo observada entre ambas puede atribuirse al backend de asignación y no a discrepancias en la lógica del algoritmo. Esta decisión fortalece la validez interna del experimento.

3.2. Versión paralela con OpenMP

La versión paralela fue construida con OpenMP. La idea consiste en repartir el recorrido de los puntos entre varios hilos para que cada uno procese una fracción independiente del conjunto de datos. La distribución del trabajo se realiza con schedule(static), una elección adecuada cuando el costo por punto es relativamente homogéneo y se desea reducir el overhead del runtime.

3.2.1. Estrategia de paralelización y justificación técnica

El principal riesgo de paralelizar K-means radica en las condiciones de carrera sobre las sumas acumuladas y los conteos de cada clúster. Si todos los hilos actualizaran las mismas estructuras globales, el programa incurriría en alta contención y necesitaría mecanismos de sincronización costosos, como atomic o critical, en prácticamente cada punto.

Para evitar ese problema, se utilizaron acumuladores privados por hilo. Cada hilo mantiene sus propias sumas y conteos parciales, y solo al final de la región paralela se realiza una reducción manual hacia estructuras globales. Esta estrategia reduce la contención, mejora la escalabilidad y conserva una lógica clara y fácil de razonar desde el punto de vista de correctitud.

4. El experimento

La evaluación experimental siguió una malla de configuraciones definida para cubrir los casos exigidos por la consigna. Se trabajó con datos en dos y tres dimensiones, tamaños de entrada de 100000, 200000, 300000, 400000, 600000, 800000 y 1000000 puntos, y cuatro configuraciones de hilos: 1, la mitad de los procesadores lógicos, el número completo de procesadores lógicos y el doble de ellos. En el equipo utilizado, esto se tradujo en la malla {1, 4, 8, 16}. Cada configuración se repitió diez veces.

Los datos de entrada fueron generados de forma sintética para asegurar control sobre el número de puntos, la dimensión y la reproducibilidad. El generador distribuye los puntos alrededor de centros aleatorios mediante ruido gaussiano.

La métrica principal de comparación fue kernel_ms, ya que representa el tiempo del núcleo del algoritmo sin incluir entrada o salida. Además, se registró total_ms como complemento, con el fin de observar el costo end-to-end de una ejecución completa. Esta distinción es importante porque el objetivo del experimento no era medir únicamente el tiempo total visible por un usuario, sino evaluar si la estrategia de paralelización mejora la fase computacional dominante.

Parámetro	Valor
Dimensiones	2D y 3D
Número de puntos	100000, 200000, 300000, 400000, 600000, 800000, 1000000
Número de hilos	1, 4, 8, 16
Número de clústeres	5
Máximo de iteraciones	300
Tolerancia	1e-4
Repeticiones por configuración	10
Total de filas esperadas	700

4.1. Equipo de pruebas

La caracterización del entorno de pruebas es fundamental, ya que el rendimiento paralelo depende de factores como la cantidad de procesadores lógicos, el comportamiento del runtime de OpenMP, el sistema operativo y el compilador empleado.

Categoría	Detalle
CPU	11th Gen Intel(R) Core(TM) i7-1195G7 @ 2.90GHz
Cores físicos	4
Procesadores lógicos	8
Frecuencia máxima (MHz)	2918
Sistema operativo	Microsoft Windows 11 Home
Versión del sistema	10.0.26200
Build	26200
Arquitectura	64-bit
Compilador	gcc.exe (Rev8, Built by MSYS2 project) 15.2.0

5. Resultados experimentales

La observación más importante es que la versión paralela logró mejoras sostenidas respecto a la versión serial en prácticamente todos los escenarios de interés, con una aceleración especialmente favorable en 3D y en los mayores tamaños de entrada.

Indicador	Valor
Mejor speedup global	3.816900
Configuración del mejor speedup global	dim=3, N=1000000, k=5, threads=16
Mejor speedup en 2D	3.382782
Configuración del mejor speedup en 2D	dim=2, N=1000000, k=5, threads=16
Mejor speedup en 3D	3.816900
Configuración del mejor speedup en 3D	dim=3, N=1000000, k=5, threads=16

Dimensión	Métrica	1 hilo	4 hilos	8 hilos	16 hilos
2D	Aceleración kernel	1.010	2.220	2.659	2.862
2D	Aceleración total	1.012	1.722	1.889	1.915
3D	Aceleración kernel	0.997	2.245	2.842	3.044
3D	Aceleración total	0.995	1.541	1.688	1.727

La Tabla 3 resume los mejores casos medidos, mientras que la Tabla 4 presenta la comparación de speedup promedio por dimensión y por número de hilos. Esta segunda tabla refuerza una lectura importante: kernel_ms y total_ms cuentan historias relacionadas, pero no idénticas. El speedup sobre total_ms es menor porque incluye costos de entrada, salida y sobrecarga general del proceso que no escalan igual que el núcleo del algoritmo.

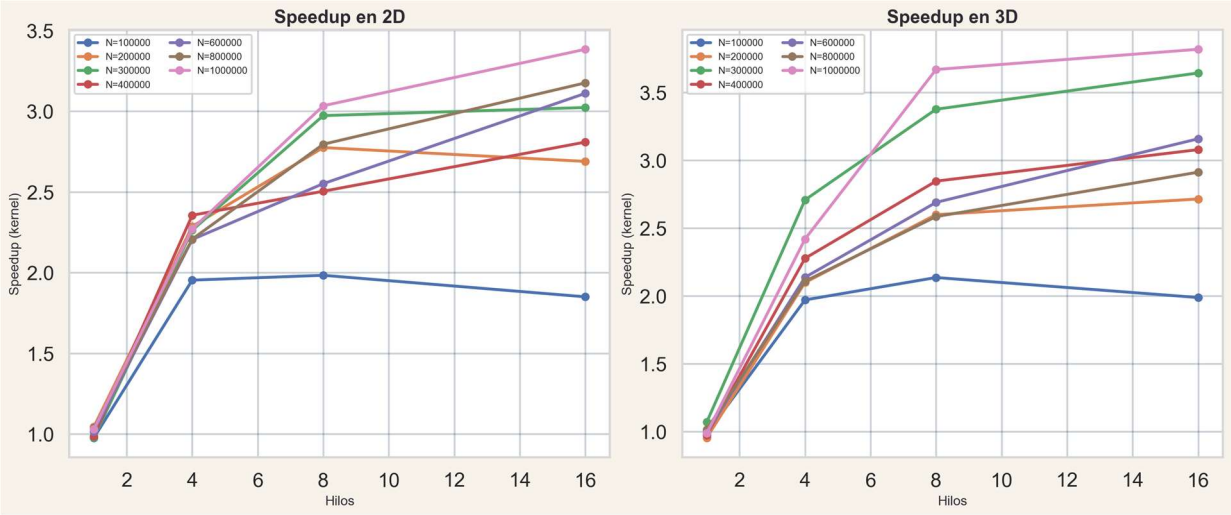

Figura 1. Vista comparativa del speedup del kernel en 2D y 3D para toda la malla experimental.

La Figura 1 concentra la lectura principal del experimento. En ambas dimensiones, el speedup crece de forma marcada entre 1 y 8 hilos, y alcanza sus mejores niveles en las entradas más grandes. La diferencia entre 2D y 3D también se aprecia visualmente: las curvas de 3D quedan un poco más arriba, lo que indica que al haber más trabajo por punto el costo extra del paralelismo se compensa mejor.

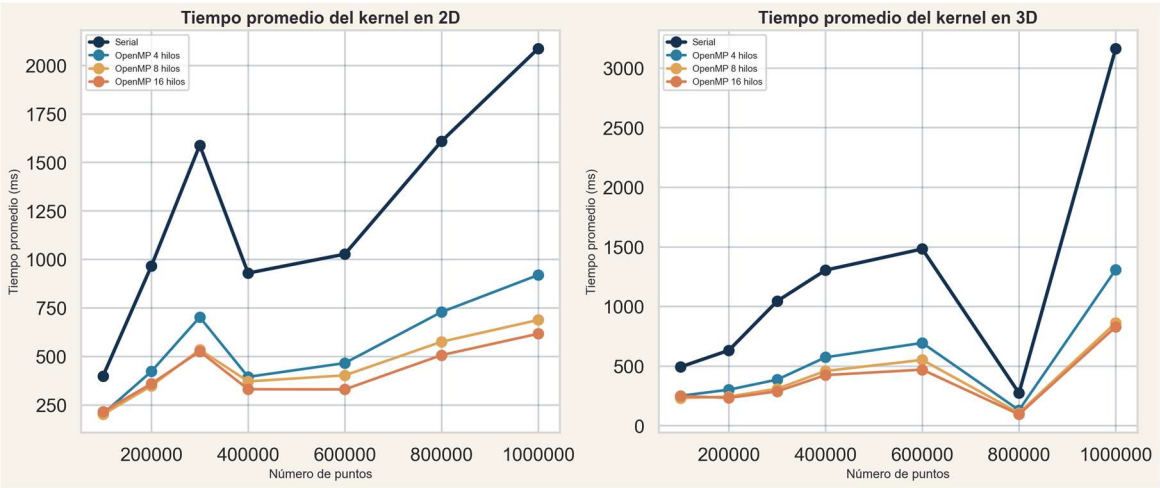

Figura 2. Comparación de tiempos promedio del kernel entre la versión serial y la versión paralela.

La Figura 2 complementa el speedup con una mirada absoluta del tiempo. En ella se observa cómo la brecha entre la línea serial y las curvas paralelas se ensancha conforme aumenta N. Esto hace visible que la mejora no es únicamente relativa, sino que también representa una reducción real y sostenida del tiempo del núcleo.

Dimensión	N	Mejor número de hilos	Speedup del mejor caso
2D	100000	8	1.9829
2D	200000	8	2.7741
2D	300000	16	3.0221
2D	400000	16	2.8081
2D	600000	16	3.1098
2D	800000	16	3.1748
2D	1000000	16	3.3828
3D	100000	8	2.1347
3D	200000	16	2.7134
3D	300000	16	3.6422
3D	400000	16	3.0772
3D	600000	16	3.1571
3D	800000	16	2.9111
3D	1000000	16	3.8169

La Tabla 5 resume el mejor número de hilos por tamaño de entrada y dimensión. Su lectura permite identificar un patrón importante: en los tamaños más pequeños, la mejor configuración no siempre es 16 hilos, mientras que en las entradas grandes la sobre-suscripción tiende a volverse favorable.

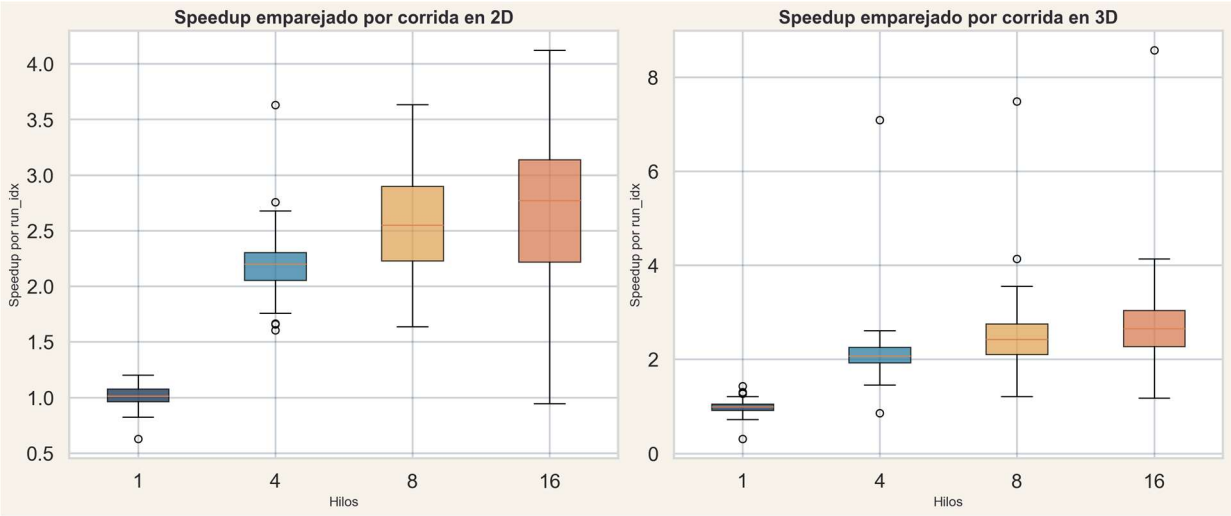

Figura 3. Distribución del speedup emparejado por corrida para cada número de hilos.

La Figura 3 hace visible la dispersión real del experimento. En ella se observa que el orden relativo entre hilos es estable, pero también que algunas configuraciones, sobre todo en 3D y tamaños grandes, presentan una variabilidad mucho mayor que otras. Esta información es crítica porque muestra que el promedio simple no describe por completo la distribución de resultados.

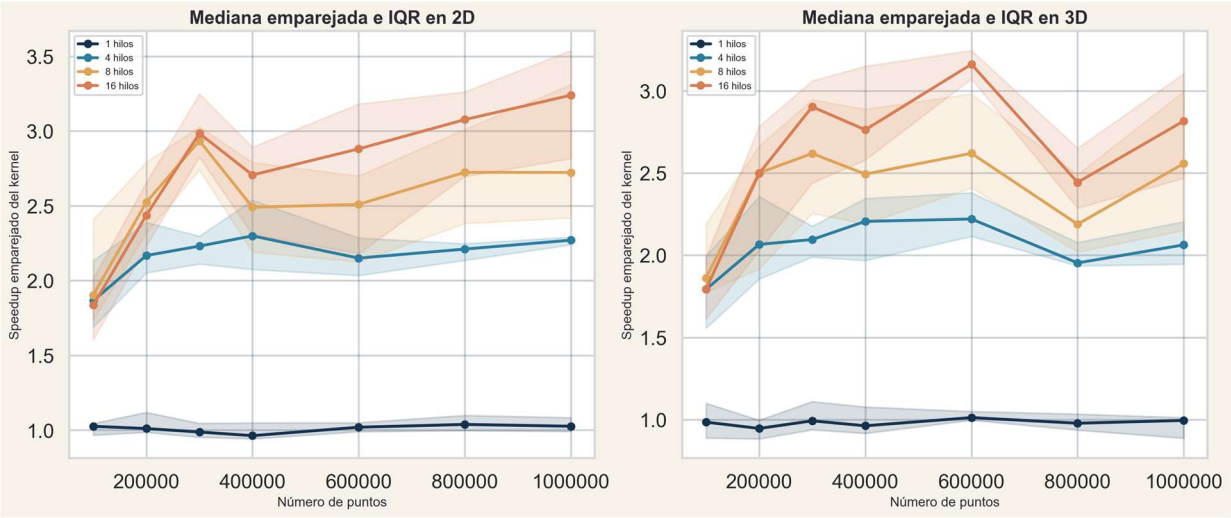

Figura 4. Evolución del speedup emparejado mediano y su rango intercuartílico conforme crece N.

La Figura 4 permite observar la transición del mejor número de hilos conforme aumenta el tamaño de entrada. La mediana emparejada crece con mayor nitidez en las entradas grandes, y la banda intercuartílica muestra de manera explícita cuándo ese crecimiento viene acompañado por una dispersión baja y cuándo se apoya en distribuciones más abiertas.

6. Análisis e interpretación

Los resultados son buenos y además son esperables. Una implementación de OpenMP saludable no debería exhibir un speedup drástico con un solo hilo, y eso es justamente lo que ocurrió: los promedios fueron 1.0099 en 2D y 0.9966 en 3D. Esto sugiere que no existe una ventaja artificial en el backend paralelo, sino un costo extra moderado asociado al runtime y a los acumuladores privados.

A medida que aumenta N, el speedup mejora de manera sostenida. Este comportamiento es coherente con uno de los principios básicos del cómputo paralelo: cuando el volumen de trabajo crece, el costo fijo de administrar hilos representa una fracción menor del tiempo total. En consecuencia, la ganancia relativa de la versión paralela se vuelve más visible.

Sin embargo, conviene distinguir entre tendencia global y magnitud puntual. El mejor speedup observado por razón de promedios fue 3.82. Cuando se utiliza una medida más robusta, como la mediana emparejada por corrida, el patrón favorable se mantiene, aunque con valores más conservadores. El mejor caso robusto del estudio fue de 3.23 en 2D con N=1,000,000 y 16 hilos. Esta diferencia no invalida el experimento; al contrario, lo hace más honesto y mejor sustentado.

El impacto del número de datos puede describirse con mayor precisión si se observa la mejor configuración por tamaño. En 2D, los tamaños de 100,000 y 200,000 puntos alcanzan su mejor desempeño con 8 hilos, mientras que a partir de 300,000 puntos la configuración de 16 hilos se vuelve dominante. En 3D, la transición es todavía más clara: solo el caso de 100,000 puntos favorece a 8 hilos, y todos los tamaños restantes benefician a 16 hilos. Esto confirma que el crecimiento de N incrementa la

relación entre trabajo útil y overhead, y por ello desplaza la mejor configuración hacia niveles mayores de paralelismo.

La reducción absoluta del tiempo también confirma esa lectura. Si en cada tamaño de entrada se toma la mejor configuración promedio observada, la disminución media del tiempo del kernel es de 64.51% en 2D y de 66.30% en 3D. Dicho de otra forma, el beneficio del paralelismo no solo se expresa como una razón abstracta de speedup, sino como un recorte sustantivo del tiempo de ejecución real.

La diferencia entre 2D y 3D también es razonable. En tres dimensiones, cada punto requiere más operaciones aritméticas para evaluar su distancia a los centroides. Por ello, la relación entre trabajo útil y overhead es ligeramente más favorable, lo que explica que el promedio con 16 hilos haya sido de 3.0436 en 3D frente a 2.8622 en 2D.

No obstante, 3D también es la dimensión donde la dispersión es más notoria. En los casos de 800,000 y 1,000,000 puntos aparecen corridas significativamente más lentas que elevan los promedios y amplían la diferencia entre media y mediana. La interpretación correcta es que 3D ofrece mayor potencial de speedup, pero también mayor sensibilidad a semillas y a condiciones de ejecución.

La estabilidad estadística puede resumirse con el rango intercuartílico del speedup emparejado. En 2D, el IQR promedio pasa de 0.0952 con 1 hilo a 0.5898 con 16 hilos. En 3D, ese mismo indicador crece de 0.1335 a 0.4992. Esto sugiere que las configuraciones más rápidas también son las más sensibles a variaciones entre corridas, especialmente cuando se combinan alta dimensionalidad y grandes tamaños de entrada.

Un punto especialmente interesante del estudio es la sobre-suscripción. En esta máquina, 16 hilos equivalen al doble de los procesadores lógicos disponibles. Aun así, la configuración de 16 hilos superó a la de 8 hilos en 5 de 7 casos en 2D y en 6 de 7 casos en 3D. Esto indica que, para esta carga y este runtime, la sobre-suscripción resultó beneficiosa en la mayoría de los escenarios, aunque no debe asumirse como una regla universal para otro hardware.

La consistencia del experimento también respalda la interpretación. No hubo discrepancias de iteraciones entre serial y OpenMP para la misma repetición, y el promedio de iteraciones fue de 101.50 en 2D y 78.19 en 3D. En otras palabras, la comparación de tiempos se realizó sobre cargas algorítmicas equivalentes y no sobre trayectorias de convergencia distintas.

Este punto es central para defender la validez del análisis: la variabilidad observada no proviene de que serial y OpenMP estén resolviendo problemas distintos dentro de una misma corrida, sino de que algunas semillas generan trayectorias de convergencia mucho más costosas que otras. Precisamente por ello, el speedup emparejado por run_idx es una herramienta metodológicamente más fuerte que el promedio simple cuando se quiere describir el comportamiento típico.

La comparación entre media y mediana permite además cuantificar el efecto de los outliers. En 2D, la brecha promedio entre el speedup basado en medias y la mediana emparejada se mantiene relativamente baja, incluso con 16 hilos, donde promedia 0.1253. En 3D la diferencia es mayor, y alcanza 0.4182 con 16 hilos y 0.4361 con 8 hilos. Esto confirma que la lectura robusta aporta valor real y no solo una reformulación del mismo resultado.

El caso con mayor diferencia entre la lectura basada en promedios y la lectura robusta se dio en 3D con N=1,000,000 y 8 hilos. Allí, el speedup por promedio fue de 3.67, mientras que la mediana emparejada fue de 2.56. Esto no significa que el resultado sea incorrecto, sino que la dispersión es lo bastante alta como para exigir una lectura más cuidadosa de los datos.

No obstante, el experimento también tiene límites. Los datos son sintéticos, se evaluó una sola máquina y algunas ejecuciones alcanzaron el máximo de 300 iteraciones. Por ello, los resultados deben interpretarse como evidencia sólida para el entorno analizado, pero no como una garantía de que el mismo speedup se mantendrá idéntico en cualquier plataforma o dataset real.

7. Conclusiones

El proyecto cumplió los requisitos propuestos. Se implementó una versión serial y una versión paralela de K-means en C, se construyó una ruta de ejecución reproducible para Windows nativo, se generaron archivos CSV de salida para visualización y se llevó a cabo un experimento sistemático para comparar el rendimiento de ambas implementaciones. Las desviaciones observadas en 2D para 300,000 puntos y en 3D para 800,000 puntos, visibles en la gráfica como picos o caídas atípicas respecto a la tendencia general, indican variaciones experimentales que podrían reducirse con un mayor control de las condiciones de ejecución. Aun así, estas anomalías son puntuales y no alteran la conclusión principal, ya que el comportamiento global del experimento sigue mostrando la aceleración esperada.

El mejor speedup global alcanzó un valor de 3.82, lo que demuestra una mejora clara frente a la versión serial. Más allá del mejor caso puntual, el comportamiento promedio confirma que la estrategia de paralelización elegida es efectiva, especialmente en 3D y en entradas grandes. Por ello, puede concluirse que la solución no solo funciona correctamente, sino que logra una aceleración real, metodológicamente sustentada y coherente con la teoría del cómputo paralelo vista en clase.